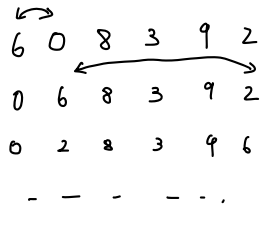
1. Sortieren.

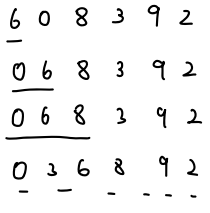
① Selection Sort. instabil



$n^2/2$

$O(n^2)$

② Insertion Sort

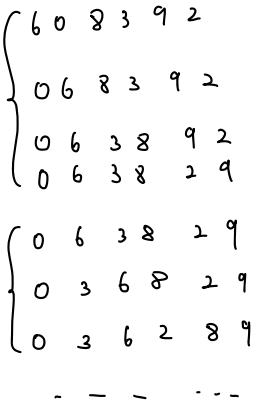


$n^2/4$

$O(n^2)$

③ Bubble Sort.

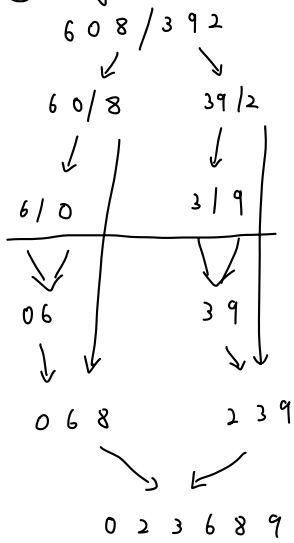
每次循环都从第一个数开始冒泡



$n^2/2$

$O(n^2)$

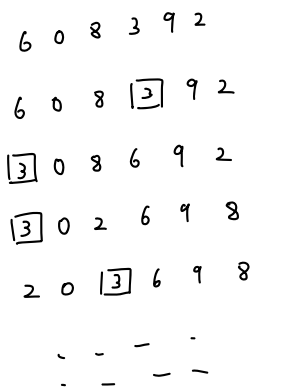
④ Merg Sort.



nlogn

$O(n\log n)$

⑤ Quick Sort. instabil

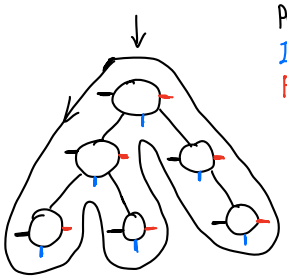


$k\,n\log n\ (k>1)$

$O(n\log n)$

	Best	Mittel	Schlechte
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$
Quick Sort	$O(n\log n)$	$O(n\log n)$	$O(n^2)$

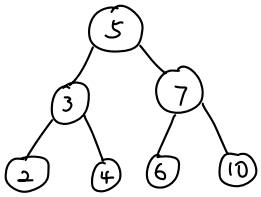
Baum.



Preorder
Inorder
Postorder

Suchbaum.

左边永远比右边小



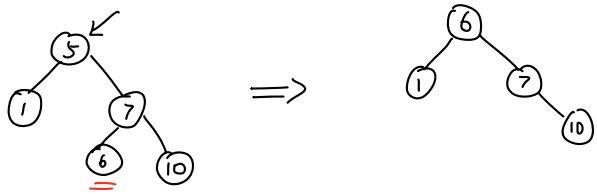
Einfügen 就只能在后面加.

Löschen.: 有左右两子　用右树的最左节点代替.

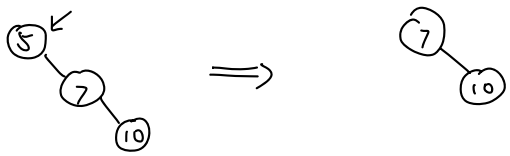
只有一子.　删了直接上移

无子.　直接删.

(1)



(2)



(3)



依有的操作都需要 Position suchen.

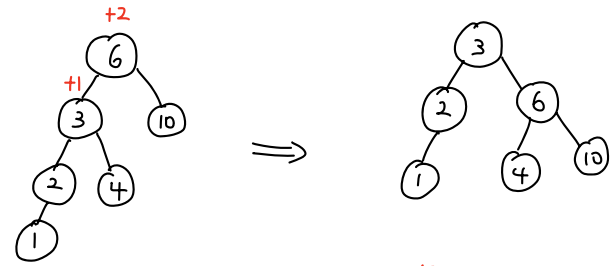
Position suchen : O(h)　h为树的高度.

对于 ausgeglichene Bäume, $h = \log_2 n$.

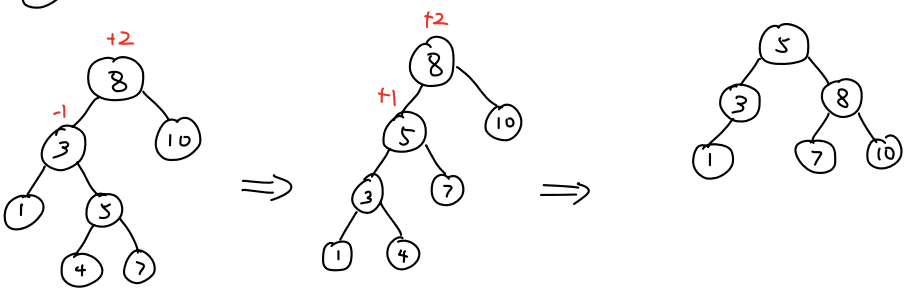
Ausgeglichene Bäume.

① AVL-Baum.

左右高度差不能到 2



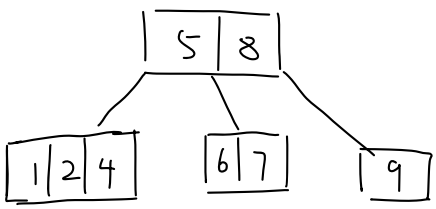
Einfügen:



Löschen: 先按照 Suchbaum 的方法删除

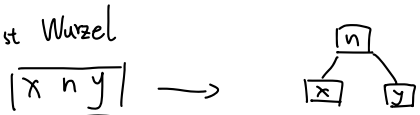
然后从下到上对每一个 Such 过的节点，做平衡

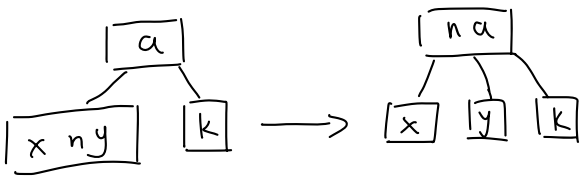
② 2-3-4 Baum.



Einfügen: 在 Such 的路上需要 split 所有 4 节点

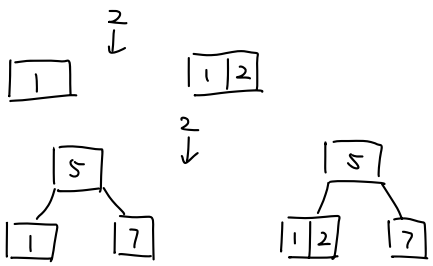
4-Knoten ist Wurzel





Einfügen 要在子节点加

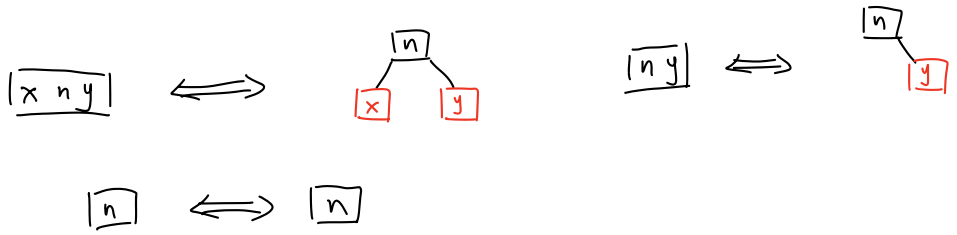
优先在同一节点，扩张.



2-3-4 Baum 的高度永远是平的.

③ Rot-Schwarz Baum.

1. Wurzel ist schwarz　　黑根.
2. rot Knoten hat schwarz Kinder　　红不连.
3. Null-Knoten sind schwarz.
4. Schwarztiefe immer gleich.　　黑深同.



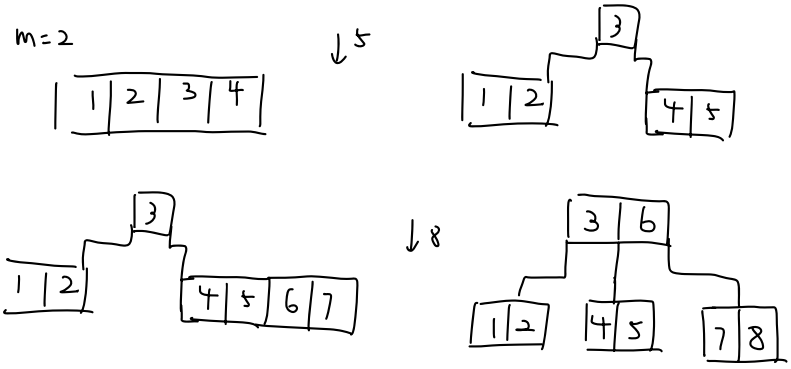
Einfügen:　默认插入红.

1. 插入为根，变黑
2. 叔是黑，以爷为节点旋转，然后旋转点与中心点变色.
3. 叔是红，叔父爷变色，爷变插入节点.

④ B-Baum.

根节点可放 1个，其余只能放 $m \sim 2m$ 个.

Einfügen: 超出限制时，中间数向上

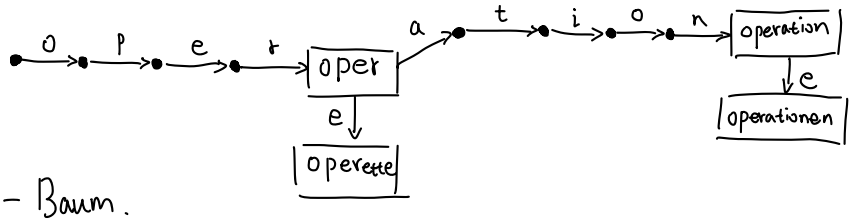


Löschen: 删中间节点则向子节点借 ⟹ 删就等于删子节点.

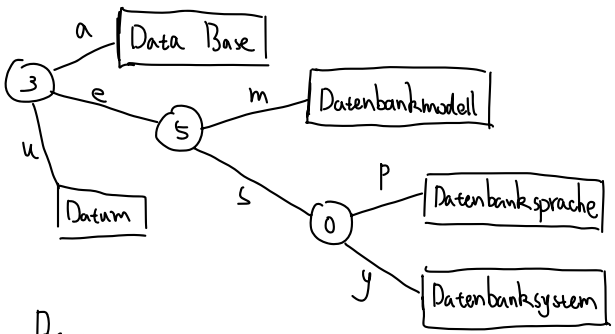
兄弟有就借兄弟的，　兄弟没多的就向上借，和兄弟合并.

Digital und Präfixbäume.

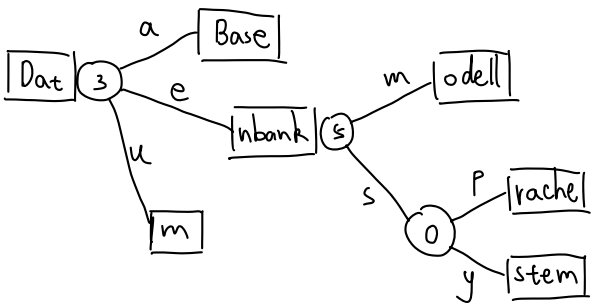
① Tries.



② Patricia – Baum.

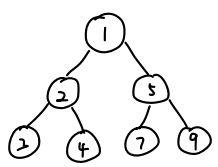


③ Präfix- Baum



Heap.

子节点比父节点大/小

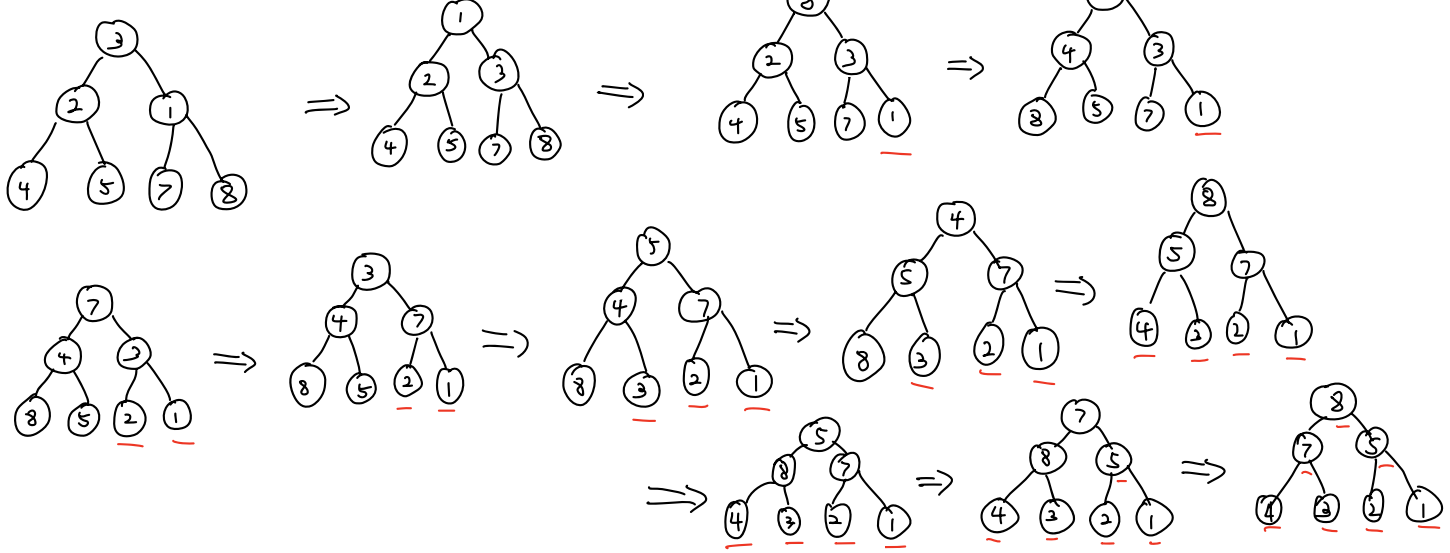


1 2 5 3 4 7 9

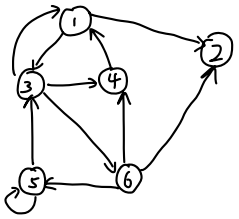
做 Heap Sort 要先整理.
从 2 开始做. 与更小的子节点交换.
一直做到没有子节点为止.

schlechtest Fall auch.

O(nlogn)



Graphen.



① Kantenliste

6, 11, 1, 2, 1, 3, 3, 1, 3, 4, 3, 6, - - - - - -

Kanten { Einfügen: $O(m)$; löschen: $O(m)$

Knoten { Einfügen: $O(1)$; löschen: $O(m)$

② Knotenliste

6, 11, 2, 2, 3, 0, 3, 1, 4, 6 - - - - - -

Kanten { $O(m+n)$; $O(m+n)$

Knoten { $O(1)$; $O(m+n)$

③ Adjazenzmatrizen.

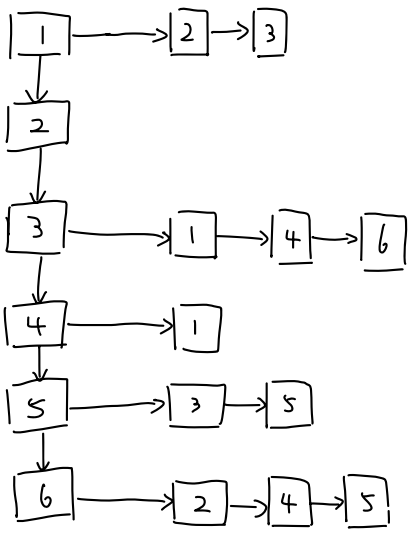
$$G_g = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

symmetrisch => ungerichtet Graphen

Kanten { $O(1)$; $O(1)$

Knoten { $O(n^2)$; $O(n^2)$

④ Adjazenzliste



Kanten { $O(n)$; $O(n)$

Knoten { $O(1)$; $O(n+m)$

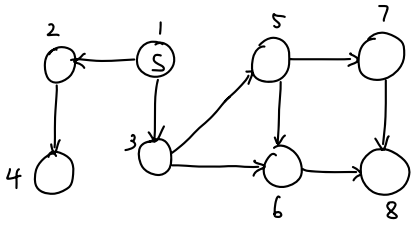
Zusammenhangskomponente.

Starke Zusammenhangskomponente → 可循环的

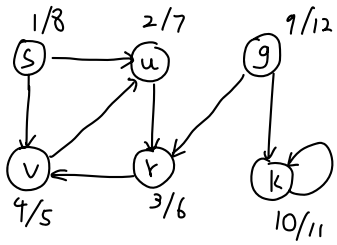
Schwache Zusammenhangskomponente → 不可循环的.

Breitensuche.

(Level Order). 从一个节点开始.

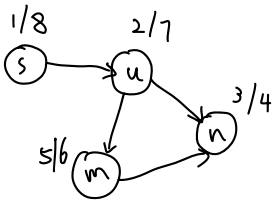


Tiefen suche



v	s
r	6
u	7
s	8
k	11
g	12

✳



Topologisches Sortieren.

用 Tiefen Suche. 大数排在前.

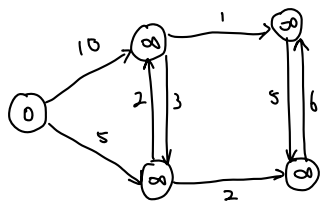
Euler'sche / Hamilton'sche Kreis / Weg.

Euler'sche : 过所有的 Kanten (每个Knoten都有偶的Grad ⇒ Euler'sche Kreis)

Hamilton : 过所有的 Knoten.

Dijkstra 算法. (找到起点到各点的最短距离, 不含负距离)

① 初始化 起点为0. 其它为∞.



② 更新相邻点的距离 (替换成更小的)

然后将最小距离点, 设为新插入点.

每次都将插入点, 排除 Queue 外. 即. 距离不能再改变了.

A* 算法. (找出从起点到终点的最短路径)

$f = g + h$.

$\begin{cases} g: \text{从起点到当前节点的实际距离} \\ h: \text{从当前节点到目标点的估计距离} \end{cases}$

① 初始化. 起点 $g = 0 \Rightarrow f = h$.

open list. 放入起点.

closed list 空.

② 从 open list 中选取 f 值最小作为当前节点.

对当前节点进行如下处理.

移入 closed list 中.

计算邻居的 g 与 f.

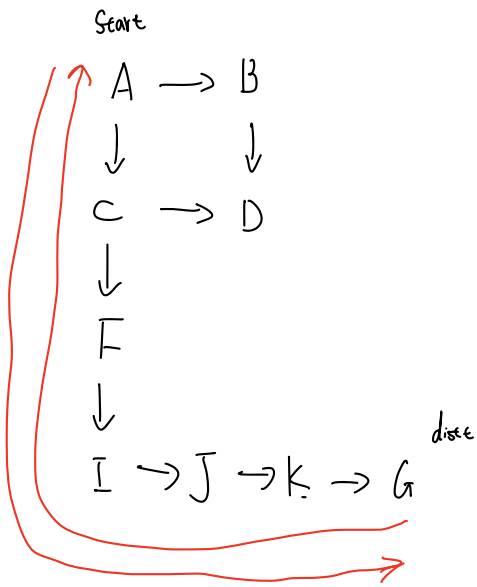
将邻居放入 open list 中 (如果它不在 open list 中)

更新邻居在 open list 中的 g 与 f.

如果邻居已在 closed list 中. 跳过该邻居.

直到当前节点为目标节点.

最终路径为. 从目标节点开始. 的通道.



Bellman – Ford 算法. (找出从起点到各点的最短距离)

可处理负距离, 但不可处理负循环.

① 初始化. 起点为 0. 其余为 ∞

② 按一定的顺序遍历所有的 Kanten.

更新所有的距离.

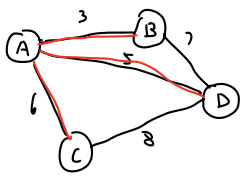
共做 |V|-1 次, V 为 Knoten 数量.

如果 |V|-1 次后, 仍然可以更新, 则是出现了负循环.

Minimale Spannbäume. (Algorithmus von Kruskal)

① 将 Kanten 根据 Gewicht 从小到大排列.

② 从小到大连成这些边, 如形成循环, 则跳过.



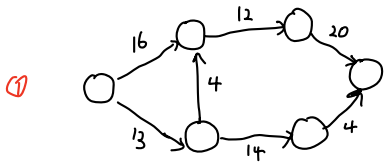
+ A B 3
+ A D 5
+ A C 6
- B D 7
- C D 8

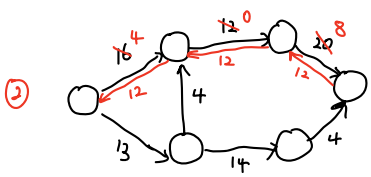
Maximaler Fluss. (Ford-Fulkerson Algorithmus)

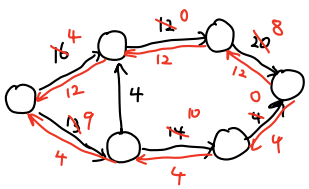
求一点到另一点的最大流量.

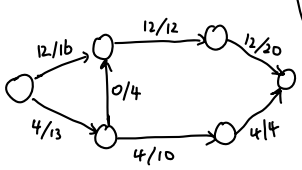
① 找到一点到另一点的所有路径.

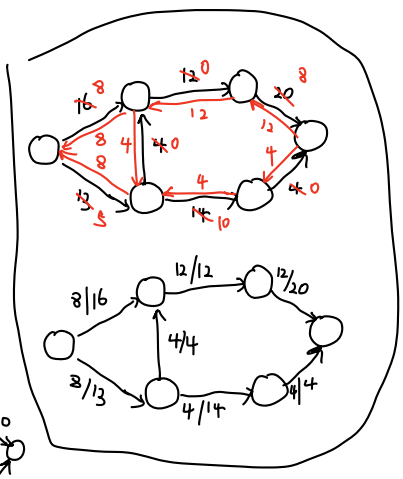
② 按一定顺序遍历这些路径.











Textalgrithmus. Suchen.

① Brute-Force-Algorithmus.
暴力算法.

② Knuth-Morris-Pratt-Algorithmus

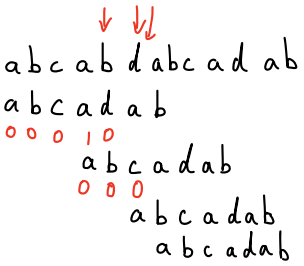
} 从前向后验证的算法.

① 写 next 表

	a	b	c	a	d	a	b
next	0	0	0	1	0	1	2

② 从错的那一位
查询 next 表的前一位. next=a

③ 从错的那位, 跳过 a 个字继续搜索.



next 表有时也叫 Längstes Präfix 表.

③ Boyer-Moore, 从后向前验证的算法.

Good-Suffix.

	A	C	B	A	B	C	B	A
	1	0	0	3	0	0	0	8

(_ _ _ _ _ a _ _)

Suffix 与 shift 计算不做要求.

last 表 属于 text.

① 先计算 last 表.

A	B	C	D
7	6	5	-1

(最右侧该字母前有多少字母)

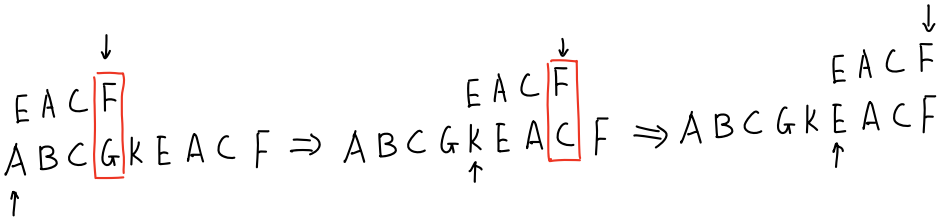
② Pos shift 表计算.

③ 向后 j-last, 如 j-last 为负 ⇒ 移 shift

j 0 1 2 3
E A C F

Last A B C E F G K
1 -1 2 0 3 -1 -1

Bad-Char



在错误产生时, 后移这一位的 j-last

如果 j-last 为负, 后移 1 位.

Text . Ähnlichkeit .

	ε	Q	u	a	n	t	i	t	ä	t
ε	0	1	2	3	4	5	6	7	8	9
Q	1	0	1	2	3	4	5	6	7	8
u	2	1	0	1	2	3	4	5	6	7
a	3	2	1	0	1	2	3	4	5	6
l	4	3	2	1	1	2	3	4	5	6
i	5	4	3	2	2	2	2	3	4	5
t	6	5	4	3	3	2	3	2	3	4
ä	7	6	5	4	4	3	3	3	2	3
t	8	7	6	5	5	4	4	3	3	2

- Initialisierung:
  $D_{0,0} = 0$; $D_{i,0} = i$; $D_{0,j} = j$
- Für $1 \leq i \leq m$, $1 \leq j \leq n$:

$$D_{i,j} = min \begin{cases} D_{i-1,j-1} + 0 & \textit{falls gleiches Zeichen} \\ D_{i-1,j-1} + 1 & \textit{falls Zeichen ersetzen} \\ D_{i,j-1} + 1 & \textit{(Zeichen einfügen)} \\ D_{i-1,j} + 1 & \textit{(Zeichen löschen)} \end{cases}$$

Hashfunktion .

Array der Größe p

h(s) = s mod p .

p是哈希表的大小.
不是数据的个数.

Kollisionsstrategien .

① Offenes Hashing .

Index	Eintrag
0	20
1	11
2	42 , 12
3	13
4	64 , 34

② Geschlossenes Hashing .

Index	Eintrag
0	
1	11 ← 21
2	
3	
4	

$G(s,i) = (h(s) + i \cdot c) \bmod p$.

$G(s,i) = (h(s) + i^2) \bmod p$.

Algorithmenmuster.

① Inkrementelles Vorgehen.

一步一步完成. (InsertionSort)

② Greedy 贪心算法.

Lokales Optimum.

(Dijkstra - Algorithmus)

③ Divide - and - Conquer.

Rekursive Rückführung.

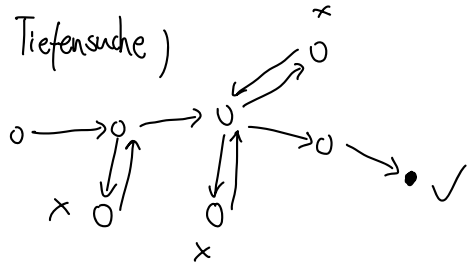
(Binäre Suche)

(QuickSort)

(MergSort)

④ Backtricking

(Tiefensuche)



⑤ Dynamische Programmierung

(Text ähnlichkeit)

小问题用于大解.

⑥ Randomisierte Algorithmen.

Las-Vegas　　Monte-Carlo

有错误率.

Thread

Synchronized + ( ) → 可以是变量 可以是方法.

防止 Thread 交插使用. 产生错误.

Semaphore. Kontrolle von gesamt Ressourcen.

up and down.

没有这个的方法, 可以同时被不同 Thread 调用